// Annotations · Configuration · REST · Data · Security · Scheduling · Testing

Core Annotations | REST / Web | Data / JPA | Configuration | Scheduling | Security | Validation | Events | Testing | Actuator | WebFlux / Reactive | Spring AI

CORE Core Bootstrap Annotations

@SpringBootApplication

entry point

```
@SpringBootApplication  // = @Configuration + @EnableAutoConfiguration + @ComponentScan
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

Component Stereotypes

bean declarations

```
@Component    // generic bean
@Service      // business logic layer
@Repository   // data layer (+ exception translation)
@Controller   // MVC controller
@RestController // @Controller + @ResponseBody
```

@Bean & @Configuration

manual bean wiring

```
@Configuration
public class AppConfig {

    @Bean
    public ObjectMapper objectMapper() {
        return new ObjectMapper();
```

```
    }
}
```

Dependency Injection

wiring beans

```
// Constructor injection (preferred)
@Service
public class OrderService {
    private final OrderRepo repo;
    public OrderService(OrderRepo repo) { this.repo = repo; }
}

// Field injection (avoid in prod)
@Autowired private MyService svc;

// Qualifier when multiple impls exist
@Qualifier("primaryDataSource") DataSource ds;
```

@Scope

bean lifecycle

```
@Scope("singleton")   // default
@Scope("prototype")   // new instance per injection
@Scope("request")     // per HTTP request
@Scope("session")     // per HTTP session

@Lazy  // initialize only when first needed
@Primary  // preferred bean when multiple exist
```

Lifecycle Hooks

init / destroy

```
@PostConstruct
public void init() { // runs after bean created }

@PreDestroy
public void cleanup() { // runs before bean destroyed }

// Or via @Bean(initMethod, destroyMethod)
@Bean(initMethod = "start", destroyMethod = "stop")
public MyService myService() { ... }
```

WEB REST Controller & Mapping

Request Mappings

HTTP verbs

```
@RequestMapping("/api")        // class level prefix
@GetMapping("/users")
@PostMapping("/users")
@PutMapping("/users/{id}")
@PatchMapping("/users/{id}")
@DeleteMapping("/users/{id}")
```

Parameter Extraction

binding inputs

```
@PathVariable Long id            // /users/{id}
@RequestParam String name        // ?name=foo
@RequestParam(required=false, defaultValue="0") int page
@RequestBody UserDto dto         // JSON body → object
@RequestHeader("Authorization") String token
@CookieValue("session") String sid
```

ResponseEntity

control HTTP response

```
return ResponseEntity.ok(body);
return ResponseEntity.status(201).body(created);
return ResponseEntity.noContent().build();
return ResponseEntity.notFound().build();
return ResponseEntity
    .created(URI.create("/users/" + id))
    .body(saved);
```

Exception Handling

global error handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleNotFound(Exception ex) {
        return new ErrorResponse(ex.getMessage());
    }
}
```

CORS

cross-origin config

```
// Method level
@CrossOrigin(origins = "http://localhost:3000")

// Global in WebMvcConfigurer
registry.addMapping("/api/**")
        .allowedOrigins("*")
        .allowedMethods("GET", "POST")
        .allowedHeaders("*");
```

Filter / Interceptor

request pipeline

```
// Filter
@Component
public class LogFilter implements Filter {
    public void doFilter(ServletRequest req,
                         ServletResponse res,
                         FilterChain chain) { ... }
}

// Interceptor
@Component
public class AuthInterceptor implements HandlerInterceptor {
    public boolean preHandle(HttpServletRequest req, ...) { ... }
}
```

DATA JPA / Data Access

Entity Mapping

JPA annotations

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 100)
    private String email;
```

```
    @CreatedDate @UpdateTimestamp
    private LocalDateTime updatedAt;
}
```

Relationships

@OneToMany etc.

```
@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "profile_id")
private Profile profile;

@OneToMany(mappedBy = "user", fetch = FetchType.LAZY)
private List<Order> orders;

@ManyToOne
@JoinColumn(name = "dept_id")
private Department department;

@ManyToMany
@JoinTable(name = "user_roles",
  joinColumns = @JoinColumn(name="user_id"),
  inverseJoinColumns = @JoinColumn(name="role_id"))
private Set<Role> roles;
```

Repository Pattern

Spring Data JPA

```
// Basic CRUD
public interface UserRepo extends JpaRepository<User, Long> {}

// Derived query methods
List<User> findByEmailAndActive(String email, boolean active);
Optional<User> findByUsername(String username);
long countByDepartment(Department dept);
void deleteByEmail(String email);
```

@Query

JPQL / native

```
@Query("SELECT u FROM User u WHERE u.email = :email")
User findByEmail(@Param("email") String email);

// Native SQL
@Query(value = "SELECT * FROM users WHERE status = ?1",
       nativeQuery = true)
List<User> findByStatus(String status);
```

```
// Modifying
@Modifying @Transactional
@Query("UPDATE User u SET u.active=false WHERE u.id=:id")
void deactivate(@Param("id") Long id);
```

@Transactional

transaction management

```
@Transactional  // defaults: propagation=REQUIRED, isolation=DEFAULT
@Transactional(readOnly = true)  // for read-only methods
@Transactional(propagation = Propagation.REQUIRES_NEW)
@Transactional(isolation = Isolation.SERIALIZABLE)
@Transactional(rollbackFor = Exception.class)
@Transactional(timeout = 30)
```

Pagination & Sorting

Pageable

```
Pageable pageable = PageRequest.of(0, 10,
    Sort.by("createdAt").descending());

Page<User> page = repo.findAll(pageable);
page.getContent();     // list of items
page.getTotalElements(); // total count
page.getTotalPages();    // page count
page.hasNext();         // more pages?
```

CONFIG Configuration & Properties

@Value

inject properties

```
@Value("${app.name}")
private String appName;

@Value("${app.timeout:30}")   // default = 30
private int timeout;

@Value("${app.admins}")        // comma-separated list
private List<String> admins;
```

```
@Value("#{systemProperties['java.home']}")  // SpEL
private String javaHome;
```

@ConfigurationProperties

typed config binding

```
@ConfigurationProperties(prefix = "app")
@Component
public class AppProperties {
    private String name;
    private int timeout;
    private Database database;
    // getters + setters
}
// application.yml:
// app.name=MyApp
// app.database.url=jdbc:...
```

@Profile

environment-specific beans

```
@Profile("dev")     // only active in dev
@Profile("!prod")  // active when NOT prod
@Profile({"dev", "test"})  // either profile

// Activate via:
// application.properties: spring.profiles.active=dev
// CLI: --spring.profiles.active=prod
// Env var: SPRING_PROFILES_ACTIVE=prod
```

@Conditional

conditional beans

```
@ConditionalOnProperty(name="feature.enabled", havingValue="true")
@ConditionalOnMissingBean(MyService.class)
@ConditionalOnClass(name = "com.example.SomeClass")
@ConditionalOnBean(DataSource.class)
@ConditionalOnWebApplication
@ConditionalOnExpression("'${env}'=='prod'")
```

SCHEDULING Task Scheduling

@Scheduled

cron / fixed rate

```
@EnableScheduling  // on @SpringBootApplication or @Configuration

@Scheduled(fixedRate = 5000)           // every 5s
@Scheduled(fixedDelay = 5000)          // 5s after last completes
@Scheduled(initialDelay = 1000, fixedRate = 5000)
@Scheduled(cron = "0 0 * * * *")       // every hour
@Scheduled(cron = "0 0 9 * * MON-FRI") // 9am weekdays
public void task() { ... }
```

Cron Expression Format

6-field Spring cron

```
// second minute hour day-of-month month day-of-week
"0 0 12 * * ?"       // noon every day
"0 15 10 ? * 6L"     // 10:15am last Fri of month
"0 0/5 * * * ?"      // every 5 minutes
"0 0 0 1 1 ?"        // midnight Jan 1st
"${cron.expression}" // from properties
```

@Async

async task execution

```
@EnableAsync  // on config class

@Async
public CompletableFuture<String> fetchData() {
    return CompletableFuture.completedFuture("result");
}

@Async("myThreadPool")  // named executor
public void asyncTask() { ... }

// Define custom executor
@Bean("myThreadPool")
public Executor taskExecutor() {
    return new ThreadPoolTaskExecutor();
}
```

SECURITY Spring Security

SecurityFilterChain

Spring Security 6+

```java
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .csrf(AbstractHttpConfigurer::disable)
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
        .build();
}
```

Method Security

annotation-based authz

```java
@EnableMethodSecurity  // on config class

@PreAuthorize("hasRole('ADMIN')")
@PostAuthorize("returnObject.owner == authentication.name")
@PreFilter("filterObject.active == true")
@PostFilter("filterObject.owner == authentication.name")
@Secured({"ROLE_ADMIN", "ROLE_MANAGER"})
@RolesAllowed("ADMIN")
```

UserDetails / PasswordEncoder

auth setup

```java
// PasswordEncoder bean
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

// UserDetailsService
@Override
public UserDetails loadUserByUsername(String username) {
    User u = repo.findByUsername(username)
        .orElseThrow(() -> new UsernameNotFoundException("..."));
    return User.builder().username(u.getEmail())
        .password(u.getPassword()).roles("USER").build();
}
```

VALIDATION Bean Validation

Constraint Annotations

javax/jakarta.validation

```
@NotNull            // not null
@NotBlank           // not null and not whitespace
@NotEmpty           // not null and not empty
@Size(min=2, max=50)
@Min(1) @Max(100)
@Email
@Pattern(regexp = "^[A-Z].*")
@Positive @PositiveOrZero
@Future @FutureOrPresent @Past
```

@Valid / @Validated

trigger validation

```
// Controller method
@PostMapping("/users")
public ResponseEntity<User> create(@Valid @RequestBody UserDto dto,
                                   BindingResult result) {
    if (result.hasErrors()) { ... }
}

// Service layer with groups
@Validated(OnCreate.class)
public User create(@Valid UserDto dto) { ... }
```

EVENTS Application Events

Publish / Listen

event-driven beans

```
// Custom event
public class UserCreatedEvent extends ApplicationEvent {
    public UserCreatedEvent(Object source) { super(source); }
}

// Publish
@Autowired ApplicationEventPublisher publisher;
publisher.publishEvent(new UserCreatedEvent(this));
```

```
// Listen
@EventListener
public void onUserCreated(UserCreatedEvent event) { ... }

@EventListener
@Async
public void asyncListener(UserCreatedEvent event) { ... }
```

Built-in App Events

lifecycle events

```
@EventListener(ApplicationReadyEvent.class)
public void onReady() { // app fully started }

@EventListener(ContextRefreshedEvent.class)
public void onRefresh() { ... }

@EventListener(ContextClosedEvent.class)
public void onShutdown() { ... }

// Or via CommandLineRunner / ApplicationRunner
@Bean
public CommandLineRunner init() {
    return args -> { /* startup logic */ };
}
```

TESTING Testing Annotations

Test Slices

focused context loading

```
@SpringBootTest                    // full context
@WebMvcTest(UserController.class)  // only MVC layer
@DataJpaTest                       // only JPA/repo layer
@JsonTest                          // only JSON serialization
@MockBean                          // mock a Spring bean
@SpyBean                           // spy on a real bean
```

MockMvc

test REST endpoints

```
@WebMvcTest(UserController.class)
class UserControllerTest {
    @Autowired MockMvc mockMvc;
```

```
    @MockBean UserService service;

    @Test
    void getUser() throws Exception {
        mockMvc.perform(get("/users/1"))
               .andExpect(status().isOk())
               .andExpect(jsonPath("$.name").value("Naresh"));
    }
}
```

@DataJpaTest

repository testing

```
@DataJpaTest
class UserRepoTest {
    @Autowired UserRepo repo;
    @Autowired TestEntityManager em;

    @Test
    void findByEmail() {
        em.persist(new User("test@example.com"));
        Optional<User> result = repo.findByEmail("test@example.com");
        assertThat(result).isPresent();
    }
}
```

ACTUATOR Actuator & Observability

Key Endpoints

management endpoints

```
# application.properties
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

// Endpoints
GET /actuator/health       // app health
GET /actuator/info         // app info
GET /actuator/metrics      // metrics list
GET /actuator/env          // environment properties
GET /actuator/beans        // all beans
GET /actuator/mappings     // all routes
POST /actuator/loggers     // change log levels
```

Custom Health Indicator

extend health checks

```
@Component
public class ExternalApiHealth implements HealthIndicator {

    @Override
    public Health health() {
        boolean up = checkExternalApi();
        return up
            ? Health.up().withDetail("api", "reachable").build()
            : Health.down().withDetail("api", "unreachable").build();
    }
}
```

Logging Config

log levels

```
# application.properties
logging.level.root=WARN
logging.level.com.myapp=DEBUG
logging.level.org.springframework.web=INFO
logging.level.org.hibernate.SQL=DEBUG       // show SQL
logging.level.org.hibernate.type=TRACE      // show params

// In code
private static final Logger log =
    LoggerFactory.getLogger(MyClass.class);
// Or Lombok: @Slf4j → use log.info(...)
```

WEBFLUX Spring WebFlux / Reactive

Mono & Flux Basics

reactive types

```
// Mono = 0 or 1 item
Mono<String> mono = Mono.just("hello");
Mono<User>   monoUser = Mono.empty();
Mono<User>   monoErr  = Mono.error(new RuntimeException());

// Flux = 0..N items
Flux<String> flux = Flux.just("a", "b", "c");
```

```
Flux<User>   users = Flux.fromIterable(list);
Flux<Long>   ticks = Flux.interval(Duration.ofSeconds(1));
```

Reactive Controller

@RestController with Flux/Mono

```
@RestController
@RequestMapping("/api/users")
public class UserController {

    @GetMapping("/{id}")
    public Mono<User> getById(@PathVariable Long id) {
        return service.findById(id);
    }

    @GetMapping(produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<User> streamAll() {  // SSE stream
        return service.findAll();
    }

    @PostMapping
    public Mono<ResponseEntity<User>> create(@RequestBody Mono<UserDto> dto) {
        return dto.flatMap(service::save)
                  .map(u -> ResponseEntity.status(201).body(u));
    }
}
```

Operators Cheatsheet

most-used operators

```
// Transform
mono.map(u -> u.getName())           // sync transform
mono.flatMap(u -> repo.save(u))      // async transform → Mono
flux.flatMap(u -> enrich(u))          // async, unordered
flux.concatMap(u -> enrich(u))        // async, ordered

// Filter / Collect
flux.filter(u -> u.isActive())
flux.take(10).skip(5)
flux.collectList()                    // Flux → Mono<List>
flux.count()                          // Flux → Mono<Long>

// Error handling
mono.onErrorReturn(defaultValue)
mono.onErrorResume(e -> fallback(e))
```

```
mono.doOnError(e -> log.error("{}", e))
mono.retry(3)
```

Reactive Repository

R2DBC / MongoDB

```
// R2DBC (reactive relational DB)
public interface UserRepo extends ReactiveCrudRepository<User, Long> {
    Flux<User>  findByDepartment(String dept);
    Mono<User>  findByEmail(String email);
    Mono<Long>  countByActive(boolean active);
}

// MongoDB reactive
public interface ProductRepo extends ReactiveMongoRepository<Product, String> {}
```

WebClient

reactive HTTP client

```
WebClient client = WebClient.builder()
    .baseUrl("https://api.example.com")
    .defaultHeader(HttpHeaders.CONTENT_TYPE, "application/json")
    .build();

// GET → Mono
Mono<User> user = client.get()
    .uri("/users/{id}", 42)
    .retrieve()
    .bodyToMono(User.class);

// GET → Flux (streaming list)
Flux<User> users = client.get()
    .uri("/users")
    .retrieve()
    .bodyToFlux(User.class);

// POST with body
Mono<User> created = client.post()
    .uri("/users")
    .bodyValue(dto)
    .retrieve()
    .onStatus(HttpStatus::is4xxClientError,
              r -> Mono.error(new BadRequestException()))
    .bodyToMono(User.class);
```

Functional Router

RouterFunction style

```
@Configuration
public class RouterConfig {

    @Bean
    public RouterFunction<ServerResponse> routes(UserHandler handler) {
        return route()
            .GET("/users", handler::getAll)
            .GET("/users/{id}", handler::getById)
            .POST("/users", handler::create)
            .DELETE("/users/{id}", handler::delete)
            .build();
    }
}

// Handler method signature
public Mono<ServerResponse> getById(ServerRequest req) {
    String id = req.pathVariable("id");
    return service.findById(id)
        .flatMap(u -> ServerResponse.ok().bodyValue(u))
        .switchIfEmpty(ServerResponse.notFound().build());
}
```

Combining Streams

zip / merge / combine

```
// zip: combine 2 Monos into one
Mono.zip(userMono, ordersMono)
    .map(t -> new UserWithOrders(t.getT1(), t.getT2()));

// merge: interleave two Flux streams
Flux.merge(flux1, flux2);

// concat: sequential, flux1 then flux2
Flux.concat(flux1, flux2);

// switchIfEmpty: fallback when empty
mono.switchIfEmpty(Mono.error(new NotFoundException()));

// defaultIfEmpty
mono.defaultIfEmpty(defaultUser);
```

Scheduler / Threading

control execution thread

```
// subscribeOn: where to subscribe (source thread)
flux.subscribeOn(Schedulers.boundedElastic())  // for blocking I/O
flux.subscribeOn(Schedulers.parallel())         // CPU work

// publishOn: switch thread for downstream ops
flux.publishOn(Schedulers.boundedElastic())

// Wrap blocking call safely
Mono.fromCallable(() -> blockingRepo.findAll())
    .subscribeOn(Schedulers.boundedElastic())
```

SPRING AI

Spring AI Annotations & APIs

ChatClient (fluent API)

primary AI interface

```
@Autowired ChatClient.Builder builder;

ChatClient chatClient = builder
    .defaultSystem("You are a helpful assistant.")
    .build();

// Simple call
String answer = chatClient.prompt()
    .user("What is Spring AI?")
    .call()
    .content();

// With system + user
String result = chatClient.prompt()
    .system("You are a Java expert.")
    .user(u -> u.text("Explain {topic}").param("topic", "reactive"))
    .call()
    .content();
```

Streaming Response

token-by-token streaming

```
// Streaming with Flux
Flux<String> stream = chatClient.prompt()
    .user("Explain Spring Boot in detail")
    .stream()
    .content();
```

```java
// SSE endpoint with streaming
@GetMapping(value="/chat", produces=MediaType.TEXT_EVENT_STREAM_VALUE)
public Flux<String> chat(@RequestParam String q) {
    return chatClient.prompt().user(q).stream().content();
}
```

Structured Output

JSON → POJO binding

```java
// Map to a record/class
record MovieReview(String title, String summary, int rating) {}

MovieReview review = chatClient.prompt()
    .user("Review Inception in JSON")
    .call()
    .entity(MovieReview.class);

// Map to List
List<MovieReview> reviews = chatClient.prompt()
    .user("List top 5 sci-fi movies as JSON array")
    .call()
    .entity(new ParameterizedTypeReference<>() {});
```

@AiObservation / Model Options

model config

```properties
// application.properties
spring.ai.openai.api-key=${OPENAI_API_KEY}
spring.ai.openai.chat.options.model=gpt-4o
spring.ai.openai.chat.options.temperature=0.7
spring.ai.openai.chat.options.max-tokens=1000

spring.ai.anthropic.api-key=${ANTHROPIC_API_KEY}
spring.ai.anthropic.chat.options.model=claude-sonnet-4-20250514

spring.ai.vertex.ai.gemini.project-id=${GCP_PROJECT}
spring.ai.vertex.ai.gemini.chat.options.model=gemini-2.0-flash
```

Prompt Templates

PromptTemplate / @Value

```java
// Inline template with variables
PromptTemplate pt = new PromptTemplate(
    "Summarize the following text in {language}: {text}");
```

```java
Prompt prompt = pt.create(Map.of(
    "language", "Telugu",
    "text", inputText));

// From classpath .st template file
@Value("classpath:/prompts/summarize.st")
private Resource promptResource;

PromptTemplate pt = new PromptTemplate(promptResource);
Prompt prompt = pt.create(Map.of("topic", topic));
```

Function Calling / Tools

AI tool use

```java
// Define a tool function as a @Bean
@Bean
@Description("Get current weather for a city")
public Function<WeatherRequest, WeatherResponse> getWeather() {
    return req -> weatherService.fetch(req.city());
}

// Register function in ChatClient call
String answer = chatClient.prompt()
    .user("What is the weather in Atlanta?")
    .functions("getWeather")
    .call()
    .content();

// Request/Response records
record WeatherRequest(String city) {}
record WeatherResponse(String condition, double tempC) {}
```

RAG — VectorStore

embeddings + similarity search

```java
// Ingest documents
@Autowired VectorStore vectorStore;
@Autowired TokenTextSplitter splitter;

List<Document> docs = new JsonReader(resource).get();
List<Document> chunks = splitter.apply(docs);
vectorStore.add(chunks);

// Similarity search
List<Document> results = vectorStore
    .similaritySearch(SearchRequest.query(query).withTopK(5));
```

```
// VectorStore types (via properties)
// PgVector, Redis, Chroma, Pinecone, Weaviate, Qdrant
spring.ai.vectorstore.pgvector.index-type=HNSW
spring.ai.vectorstore.pgvector.dimensions=1536
```

QuestionAnswerAdvisor

RAG advisor pattern

```
// Auto-retrieval + augmentation via Advisor
ChatClient ragClient = builder
    .defaultAdvisors(
        new QuestionAnswerAdvisor(vectorStore,
            SearchRequest.defaults()),
        new MessageChatMemoryAdvisor(chatMemory)
    )
    .build();

// Simple RAG call — context injected automatically
String answer = ragClient.prompt()
    .user(question)
    .call()
    .content();
```

Chat Memory

multi-turn conversations

```
// In-memory (for dev)
ChatMemory memory = new InMemoryChatMemory();

ChatClient statefulClient = builder
    .defaultAdvisors(new MessageChatMemoryAdvisor(memory))
    .build();

// Use conversationId to isolate sessions
String reply = statefulClient.prompt()
    .user(userMessage)
    .advisors(a -> a.param(
        AbstractChatMemoryAdvisor.CHAT_MEMORY_CONVERSATION_ID_KEY,
        sessionId))
    .call()
    .content();
```

Image & Multimodal

vision / image generation

```java
// Image generation
@Autowired ImageModel imageModel;

ImageResponse resp = imageModel.call(
    new ImagePrompt("A futuristic city in India",
        OpenAiImageOptions.builder()
            .withModel("dall-e-3")
            .withSize("1024x1024").build()));
String imageUrl = resp.getResult().getOutput().getUrl();

// Vision: pass image to chat
var media = new Media(MimeTypeUtils.IMAGE_PNG, imageResource);
String desc = chatClient.prompt()
    .user(u -> u.text("Describe this image").media(media))
    .call().content();
```

EmbeddingModel

generate embeddings

```java
@Autowired EmbeddingModel embeddingModel;

// Single embedding
float[] vector = embeddingModel
    .embed("Spring AI is powerful");

// Batch embeddings
EmbeddingResponse resp = embeddingModel.embedForResponse(
    List.of("text one", "text two"));
resp.getResults().forEach(r ->
    System.out.println(r.getOutput().length));
```

MCP Client (Tool Servers)

Model Context Protocol

```properties
# application.properties — MCP server config
spring.ai.mcp.client.toolcallback.enabled=true
spring.ai.mcp.client.stdio.servers.myServer.command=npx
spring.ai.mcp.client.stdio.servers.myServer.args=-y,@my/mcp-server
```

```java
// Auto-wired MCP tools into ChatClient
@Autowired List<McpSyncClient> mcpClients;
@Autowired ToolCallbackProvider toolCallbacks;

ChatClient mcpClient = builder
```

```
    .defaultTools(toolCallbacks)
    .build();
```